Machine Problem 1 Documentation

Machine Problem Title: Towers of Hanoi using Stacks

Course & Section: CS102-1L-CIS201

Programming Languages Used: Python

Submitted By: Noda, Isaiah Andrei I. & Olayvar, Hannali S.,

Date Submitted: 20/10/2025

1. The Problem Description

The **Towers of Hanoi** is a mathematical puzzle consisting of three towers — **A**, **B**, and **C** — and a number of disks of different sizes stacked on one tower in decreasing order (largest at the bottom, smallest at the top).

The objective of the puzzle is to move all the disks from **Tower A** (source) to **Tower C** (destination), using **Tower B** (auxiliary) as temporary storage, while following these rules:

1. Only **one disk** may be moved at a time.
2. Only the **top disk** of a tower can be moved.
3. **No larger disk** may be placed on top of a smaller disk.

This project implements the Towers of Hanoi game in **Python**, using **Stacks** that are implemented through **Linked Lists**.

The program allows the user to **interactively play** the game by typing moves such as A C, while ensuring that all moves follow the rules of the game. The towers are visually displayed using asterisks (*) that represent the disks.

2. References and Project Resources

This section provides access to the project's supporting materials, including the report, documentation, and the repository containing the source code and updates.

- Github Repository - source code
- Development progress report - additional document showing whole progress
- Simple Flowchart
- Complex Flowchart

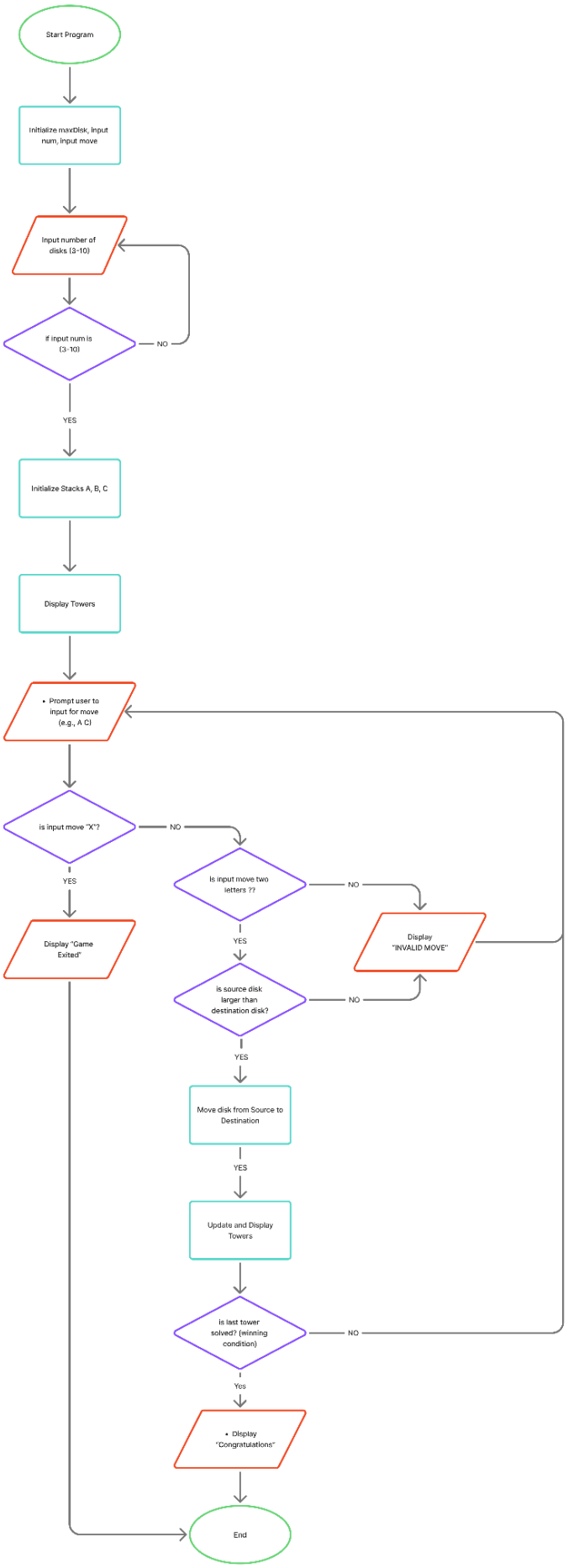
3. Program Design and Structure

Simple Flowchart

This flowchart describes the overall logic of the program. This simple flowchart shows the basic flow of the Tower of Hanoi game. It focuses on the main steps — getting the number of disks, displaying the towers, moving disks, and checking for the winning condition.

The following in the chart are functions and are considered as processes :

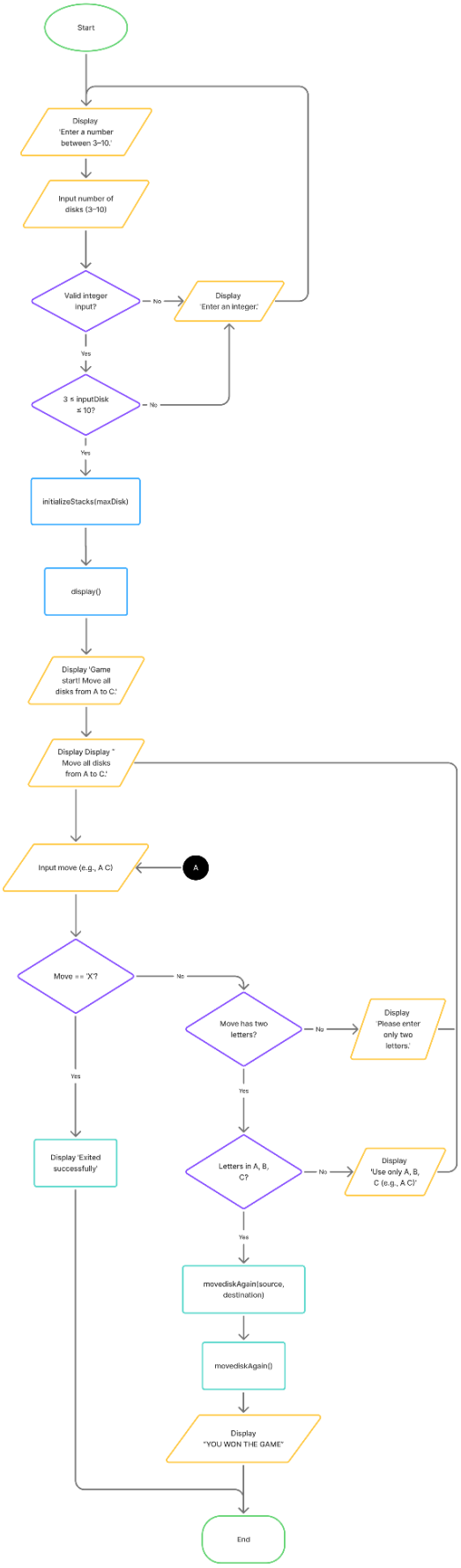
- initializeStacks()
- display()
- getLastDisk(stack)
- moveDiskAgain(src, dst)

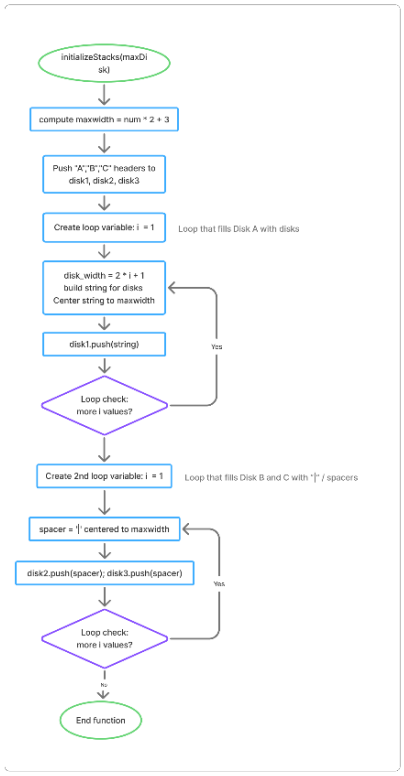


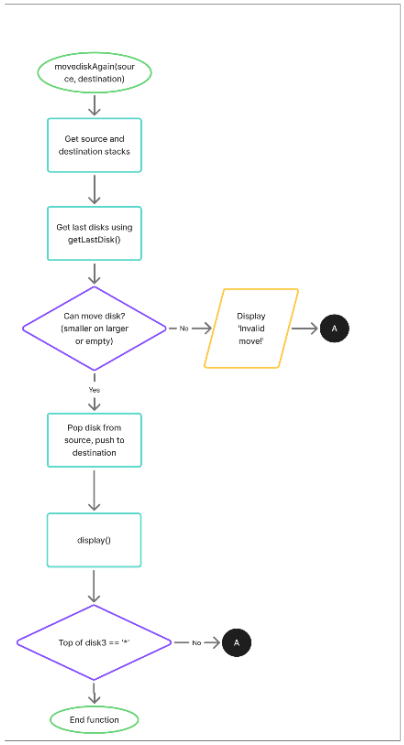
Complex Flowchart

This complex flowchart presents a detailed version of the Tower of Hanoi program. It includes the internal functions, validations, and stack operations involved in moving the disks. It explains not just what happens, but how each part of the code works behind the scenes, making it more technical and comprehensive.

- This version includes **multiple sub-processes** functions initializeStacks() moveDiskAgain()).
- It maps the **entire logic**, including how disks are stored, validated, and displayed in the towers.







Classes

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```

Attribute	Description
value	Stores the data in the data field
next	Points to the next node in the stack

- Class **Node** in **node.py**

  Description: Initialization of a single element(node) in the linked list to be used later in the stack. The Node class represents a single element in a linked list. Each node stores a value and a reference called next that points to the next node in the sequence. It serves as the basic building block for creating stacks used in the Tower of Hanoi program.

- Class **Stack** in **stack.py**

```
from node import Node

class Stack:
    def __init__(self):
        self.top = None

    def push(self, value):
        new_node = Node(value)
        new_node.next = self.top
        self.top = new_node

    def pop(self):
        if not self.is_empty():
            poppedvalue =
self.top.value
            self.top =
self.top.next
            return poppedvalue
        return None

    def is_empty(self):
        return self.top is None

    def peek(self):
        return self.top.value
```

Method	Description
__init__	Initializes an empty stack and sets the top to None.
push(value)	Adds a new item to the top of the stack
pop()	Removes the top of stack and returns its value
is_empty()	Checks if the stack is empty (no disks)
peek()	Returns top disk value without deleting

Description: The stack class initializes and manages stack objects used to represent the towers in the Towers of Hanoi game. We can perform the above mentioned stack operations for handling stack data structure using linked lists. It implements a stack data structure using linked nodes. It allows adding elements (push), removing the top element (pop), checking if the stack is empty (is_empty), and viewing the top value without removing it (peek). This class is important in the Tower of Hanoi program to manage the movement of disks between towers.

- Main **script** in **main.py**

```
from tower_logic import Stack

disk1 = Stack()
disk2 = Stack()
disk3 = Stack()
isRunnable = True
def main():
    …

def display():
   . . .
   #TRAVERSE THROUGH EACH
STACKS CURRENT TO PRINT SIDE BY
SIDE\

def initializeStacks(num:int):
   . . .
   #CREATE STACK 1 (DISK A) AND
INSERT DISKS. CREATE STACK 2
AND 3 WITH FILLER LINES "|"


def getLastDisk(stack):
   . . .
   #TRAVERSE THROUGH STACK AND
RETURN LAST DISK (FOR
VALIDATION PURPOSES)



def movediskAgain(source,
destination):
   . . .
    #INITIALIZE NEEDED
VARIABLES

    if len(src_lastdisk) <
len(dst_lastdisk) or
len(src_lastdisk) == 1 or
len(dst_lastdisk) == 1:
        . . .
        #TRAVERSE THRU SOURCE
AND POP IT
        #TRAVERSE THRU THE
DESTINATION

     #WINNING CONDITION
STATEMENT



if __name__ == "__main__":
    main()
```

Method	Description
main()	The main function that starts the program. It asks the user for the number of disks (3–10), initializes the towers, displays them, and handles user inputs for moving disks between towers.
display()	Prints the current state of the three towers (A, B, and C) side by side, showing how the disks are arranged on each tower.
initializeStacks(maxDisk)	Set up the initial state of the game. Tower A is filled with all disks (represented by *) while Towers B and C are initialized with spacers '|'
getLastDisk(stack)	Retrieves the topmost disk from a given stack without permanently removing it. It is used to check which disk is currently on top before validating or performing a move.
moveDiskAgain(source, destination)	Handles the logic of moving a disk from one tower to another. It checks if the move is valid (no larger disk can be placed on a smaller one) and updates both source and destination towers accordingly. Displays the new tower state after each move.

Description: This class implements the **Towers of Hanoi** puzzle using a stack data structure based on linked lists. It utilizes the **Stack** class from **stack.py** and the **Node** class from **node.py** to manage the data and operations needed for the game. The program's goal is to move all disks from **Tower A to Tower C**, following the classic rules: only one disk can be moved at a time, a larger disk cannot be placed on a smaller one, and Tower B serves as the auxiliary tower.

The program begins by asking the user for the number of disks (3–10), then initializes Tower A with disks represented by asterisks (*) while Towers B and C are filled with spacers (|). Each tower is represented by a stack, with the top element corresponding to the visible top disk. The display() method prints all three towers side by side after each move, while getLastDisk() retrieves the top disk of a tower for validation. The moveDiskAgain() method handles the logic for moving disks, ensuring that each move follows the puzzle's rules before updating and displaying the new tower states.

Grading rubric

	Excellent	Good	Fair	Poor
Problem description	15 Clearly and thoroughly explains the Towers of Hanoi problem; includes objectives, rules, and stack-based solution overview; well written and technically accurate.	10 Mostly clear and correct; minor omissions or unclear phrasing.	5 Basic explanation; missing some objectives or problem rules.	1 Incomplete, vague, or missing explanation of the problem.
Program design and structure	30 Provides clear and accurate flowchart; includes complete class and method descriptions with correct relationships; structure is well organized and easy to follow.	20 Flowchart and design are mostly correct; minor logical or structural inconsistencies; class/method. explanations mostly clear.	10 Flowchart incomplete or poorly labeled; class/method details are lacking.	2 Incorrect flowchart, classes, and structure

Total = 45 points